BÁO CÁO THỰC HÀNH LAB 1

Mô hình hóa An toàn Phần mềm bằng Cấu trúc Kripke (Access Control Module)

Học phần: An toàn phần mềm (CSE703093) | Hệ thống: SecureSys Access Control | Ngày thực hiện: 11/10/2026

---

1. MỤC TIÊU BÀI THỰC HÀNH

Bài thực hành tập trung vào việc áp dụng các phương pháp hình thức (Formal Methods) trong bảo đảm an toàn phần mềm, cụ thể:

1. **Hình thức hóa hệ thống:** Xây dựng mô hình toán học Cấu trúc Kripke M = (S, S0, R, L) cho phân hệ kiểm soát ra vào cửa điện tử (Access Control Module) thuộc hệ thống SecureSys.
2. **Kiểm chứng mô hình (Model Checking):** Cài đặt giải thuật duyệt không gian trạng thái (BFS) trong Python để tự động kiểm tra các thuộc tính an toàn logic thời gian CTL: Invariant Safety (AG !p), Conditional Safety (AG (p -> q)), và Reachability (EF p).
3. **Mô phỏng thực thi an toàn (Runtime Simulator):** Lập trình chương trình C thực thi máy trạng thái với các nguyên tắc phòng thủ: chống tràn bộ đệm, giải phóng bộ nhớ, và tuân thủ nguyên tắc Fail-Closed (từ chối sự kiện lạ và duy trì trạng thái an toàn).
4. **Phát hiện lỗ hổng và Vacuous Truth:** So sánh hành vi trên 3 mô hình (SAFE, BUGGY, FIXED), phát hiện đường phản ví dụ (Counterexample Trace) của lỗi logic race-condition và làm chủ hiện tượng Chân lý rỗng (Vacuous Truth) khi cảm biến bị ngắt kết nối.

2. HÌNH THỨC HÓA CẤU TRÚC KRIPKE HỆ THỐNG ACCESS CONTROL

Hệ thống khóa cửa điện tử SecureSys được đặc tả hình thức bằng cấu trúc Kripke 4 thành phần:

$$\mathbf{M = (S, S0, R, L)}$$

2.1. Tập trạng thái hữu hạn (S)

Không gian trạng thái bao gồm 5 trạng thái thiết kế hợp lệ và 1 trạng thái lỗi phát sinh trong mô hình BUGGY:

- **S0 (Locked_Idle):** Cửa đang khóa, hệ thống ở chế độ rảnh rỗi chờ người dùng quẹt thẻ.
- **S1 (Authenticating):** Đã nhận thẻ, hệ thống đang chờ người dùng nhập mã PIN và xử lý xác thực.
- **S2 (Authenticated_Unlock):** Xác thực danh tính thành công, chốt điện tử mở cửa cho phép vào.
- **S3 (Locked_Alarm):** Nhập sai mã PIN quá 3 lần, cửa khóa chặt và kích hoạt còi báo động.
- **S4 (TamperDetected_Alarm):** Cảm biến phát hiện hành vi cạy phá vật lý, khóa cửa đóng và hú còi khẩn cấp.
- **S5 (BUG_Unlock_NoAuth):** Trạng thái lỗi logic (chỉ có trong bản BUGGY) do timeout glitch làm cửa mở mà không qua xác thực.

2.2. Trạng thái khởi tạo (S0)

Hệ thống luôn khởi động an toàn từ trạng thái khóa rảnh rỗi: **S0 = { S0 }**.

2.3. Tập mệnh đề nguyên thủy (AP) và Hàm gán nhãn (L)

Tập mệnh đề nguyên thủy biểu diễn các sự kiện và tính chất tại mỗi trạng thái: **AP = { locked, unlocked, alarm, authenticated, tamper }**.

Hàm gán nhãn L: S -> 2^AP xác định các tính chất đúng tại từng trạng thái:

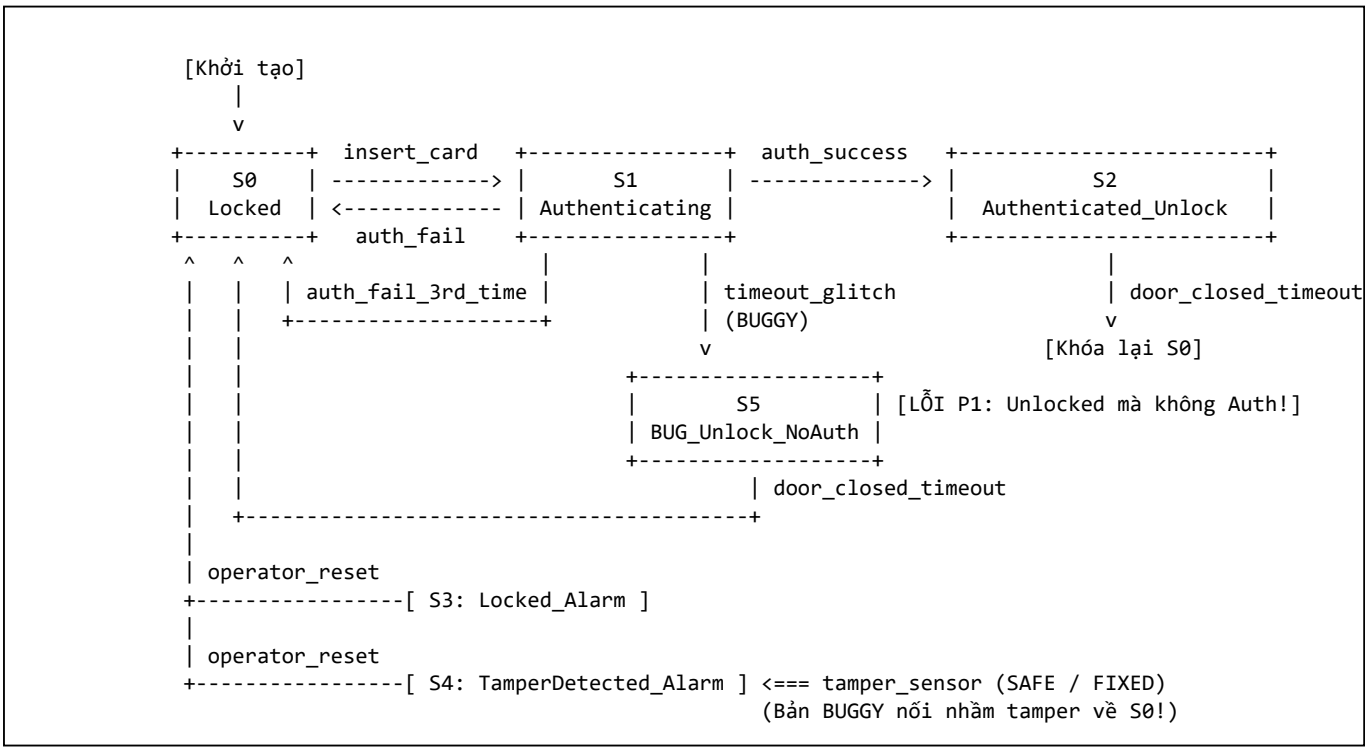
- L(S0) = { locked }
- L(S1) = { locked }
- L(S2) = { unlocked, authenticated }
- L(S3) = { locked, alarm }
- L(S4) = { locked, alarm, tamper }
- L(S5) = { unlocked } *(Điểm yếu bảo mật: trạng thái mở khóa nhưng thiếu hẳn nhãn authenticated)*

2.4. Quan hệ chuyển trạng thái (R)

Bảng tổng hợp các chuyển tiếp trạng thái theo sự kiện trong hệ thống:

Từ trạng thái	Sự kiện kích hoạt	Đích (Bản SAFE)	Đích (Bản BUGGY)	Ý nghĩa bảo mật
S0	insert_card	S1	S1	Quẹt thẻ hợp lệ, chuyển sang xác thực
S1	auth_success	S2	S2	Xác thực mã PIN đúng -> Mở cửa
S1	auth_fail	S0	S0	Sai PIN dưới 3 lần -> Trả về trạng thái chờ
S1	auth_fail_3rd_time	S3	S3	Sai PIN 3 lần -> Khóa tài khoản, báo động
S2	door_closed_timeout	S0	S0	Hết thời gian mở -> Tự động khóa lại
S3	operator_reset	S0	S0	Quản trị viên can thiệp giải tỏa báo động
S0	tamper_sensor	S4	**S0 (LỖI)**	Bản SAFE kích hoạt S4; Bản BUGGY nuốt sự kiện về S0
S4	operator_reset	S0	S0	Quản trị viên kiểm tra và reset hệ thống
S1	timeout_glitch	(Không có)	**S5 (LỖI)**	Lỗi Race Condition làm mở cửa không cần xác thực
S5	door_closed_timeout	(Không có)	S0	Hết timeout đóng cửa về S0

2.5. Sơ đồ chuyển trạng thái hệ thống



3. ĐẶC TẢ CÁC THUỘC TÍNH KIỂM CHỨNG CTL

Để đảm bảo an toàn tuyệt đối, hệ thống được kiểm chứng thông qua 5 thuộc tính logic thời gian CTL (Computation Tree Logic):

Thuộc tính	Công thức CTL	Dạng kiểm chứng	Mục tiêu bảo mật
P1	`AG !(unlocked AND NOT authenticated)`	Safety Invariant	**Ngăn chặn mở cửa trái phép:** Trong mọi trạng thái reachable, không bao giờ cửa mở mà chưa được xác thực.
P2	`AG (tamper -> alarm)`	Conditional Safety	**Phản ứng sự cố vật lý:** Hễ có phát hiện cạy phá (tamper) thì bắt buộc hệ thống phải ở trạng thái báo động (alarm).
P3	`EF (unlocked)`	Reachability (Liveness)	**Tính khả thi chức năng:** Phải tồn tại ít nhất một kịch bản hợp lệ đưa hệ thống đến trạng thái mở khóa.
P4 (Mới)	`AG !(unlocked AND alarm)`	Mutual Exclusion	**Tính loại trừ trạng thái:** Không bao giờ hệ thống vừa mở khóa cho người vào vừa bật chuông báo động trộm cướp.
P5 (Mới)	`EF (tamper)`	Reachability Guard	**Chống Chân lý rỗng (Vacuity Guard):** Trạng thái cạy phá phải thực sự có thể đạt tới được để đảm bảo P2 có ý nghĩa thực tế.

4. KIẾN TRÚC VÀ CÀI ĐẶT MÃ NGUỒN

4.1. Tầng Kiểm chứng Tĩnh (Python: `python/kripke_model.py`)

Chương trình Python tải mô hình từ tệp cấu hình JSON và duyệt đồ thị trạng thái:

- **Duyệt không gian trạng thái (BFS):** Hàm `reachable_states()` thực hiện duyệt theo chiều rộng xuất phát từ S0, đảm bảo chỉ kiểm chứng trên các trạng thái thực tế có thể đạt tới và tránh lặp vô hạn.
- **Truy vết phản ví dụ (Counterexample Generation):** Khi phát hiện trạng thái vi phạm điều kiện an toàn, hàm `_find_path()` trích xuất chuỗi trạng thái và sự kiện ngắn nhất từ S0 đến trạng thái lỗi để phục vụ vá lỗi.
- **Cơ chế tự động phát hiện Vacuous Truth:** Khi kiểm tra thuộc tính dạng kéo theo `AG (p -> q)`, chương trình đếm số lượng trạng thái reachable thỏa mãn tiền đề `p`. Nếu không có trạng thái nào có `p = True`, chương trình lập tức đưa ra cảnh báo: `[CANH BAO: VACUOUS PASS - tien de khong bao gio xay ra!]`.

4.2. Tầng Mô phỏng Thực thi An toàn (C: `c/access_simulator.c`)

Chương trình C mô phỏng máy trạng thái ở cấp độ thực thi runtime, tuân thủ các chuẩn an toàn phần mềm nghiêm ngặt:

- **Loại bỏ triệt để hàm nguy hiểm:** Tuyệt đối không dùng `gets()` hay `scanf("%s")`. Toàn bộ chuỗi được đọc qua hàm `safe_read_line()` sử dụng `fgets(buf, sizeof(buf), fp)` có kiểm soát kích thước đệm.
- **Kiểm soát biên và phòng thủ con trỏ:** Mọi thao tác tách chuỗi CSV đều có tham số `max_fields`; con trỏ tệp tin và mảng chuyển tiếp được kiểm tra điều kiện NULL và tràn kích thước tối đa.
- **Nguyên tắc Fail-Closed:** Khi nhận được sự kiện không hợp lệ tại trạng thái hiện tại, hàm chuyển trạng thái `next_state()` từ chối sự kiện, tăng biến đếm `unsafe_event_count`, in thông báo từ chối và **giữ nguyên trạng thái an toàn hiện tại** (không bao giờ chuyển bừa bãi).
- **Kiểm tra xác nhận lúc chạy (Runtime Assertion):** Sau mỗi bước chuyển tiếp, hệ thống kiểm tra ngay lập tức nếu trạng thái đích mang nhãn `unlocked` nhưng thiếu nhãn `authenticated` thì xuất cảnh báo vi phạm an toàn.

5. KẾT QUẢ THỰC NGHIỆM VÀ ĐỐI CHIẾU BẰNG CHỨNG

5.1. Kiểm chứng trên Mô hình Chuẩn (SAFE Model)

Thực thi lệnh kiểm chứng Python trên mô hình chuẩn:

```
python python/kripke_model.py dataset/access_system_model.json
```

Kết quả ghi nhận: Cả 5 trạng thái đều reachable. Thuộc tính P1 đạt chuẩn (PASS), P2 đạt chuẩn (PASS), P3 đạt chuẩn (PASS) với đường đi: `Locked_Idle -> Authenticating -> Authenticated_Unlock` qua hai sự kiện `insert_card` và `auth_success`.

5.2. Kiểm chứng trên Mô hình Lỗi (BUGGY Model) & Bóc trần Vacuous Truth

Thực thi lệnh kiểm chứng Python trên mô hình có lỗi:

```
python python/kripke_model.py dataset/access_system_model_BUGGY.json
```

Kết quả ghi nhận:

```
=== He thong: AccessControl-Door-v2-BUGGY ===
So trang thai khai bao: 6 | Reachable: 5 / 6

[P1_SAFETY_NO_UNLOCK_WITHOUT_AUTH]
  Ket qua   : VI PHAM (FAIL)
  Duong di  : Locked_Idle -> Authenticating -> BUG_Unlock_NoAuth
  Su kien   : insert_card , timeout_glitch

[P2_SAFETY_ALARM_ON_TAMPER]
  Ket qua   : THOA MAN (PASS) [CANH BAO: VACUOUS PASS - tien de khong bao gio xay ra!]

[P3_REACHABILITY_UNLOCK]
  Ket qua   : THOA MAN (PASS)
```

Phân tích chuyên sâu hiện tượng Chân lý rỗng (Vacuous Truth) tại P2:

- Trên bản BUGGY, sự kiện `tamper_sensor` bị nối nhầm lặp lại về `S0`, khiến cho trạng thái `S4` (`TamperDetected_Alarm`) bị cô lập hoàn toàn (Unreachable State, 5/6 trạng thái đến được).
- Vì S4 không thể đạt tới, trên mọi trạng thái có thể đến được của hệ thống, mệnh đề `tamper` luôn luôn nhận giá trị **False**.
- Theo luật suy diễn mệnh đề logic: **False -> alarm = True** (tiền đề sai thì mệnh đề kéo theo luôn đúng).
- Hệ quả là thuộc tính P2 báo PASS giả tạo! Trình kiểm chứng thông thường sẽ đánh lừa kỹ sư an toàn rằng hệ thống vẫn chống cạy phá tốt, trong khi thực tế cảm biến báo động đã bị tê liệt hoàn toàn.

5.3. Kết quả Mô phỏng Thực thi C (Runtime Verification)

Chương trình C (`access_simulator.exe`) chạy trên 3 kịch bản thực tế chứng minh tính an toàn runtime:

Kịch bản 1: Luồng nghiệp vụ chuẩn trên mô hình SAFE

```
./c/access_simulator.exe dataset/access_safe_states.csv dataset/access_safe_transitions.csv dataset/ac
cess_safe_init.txt insert_card auth_success door_closed_timeout

Log: S0 (Locked_Idle) --(insert_card)--> S1 --(auth_success)--> S2 (Authenticated_Unlock) --(door_clos
ed_timeout)--> S0
Trang thai cuoi cung: S0 | So su kien bi tu choi: 0
```

Kịch bản 2: Bắt lỗi Runtime vi phạm P1 trên mô hình BUGGY

```
./c/access_simulator.exe dataset/access_buggy_states.csv dataset/access_buggy_transitions.csv dataset/
access_buggy_init.txt insert_card timeout_glitch

Log: S0 --(insert_card)--> S1 --(timeout_glitch)--> S5 (BUG_Unlock_NoAuth) [unlocked]
[CANH BAO AN TOAN] Trang thai 'S5' la 'unlocked' nhung KHONG 'authenticated' - vi pham thuoc tinh P1!
Trang thai cuoi cung: S5 | Khop 100% voi vet phan vi du cua Python.
```

Kịch bản 3: Kiểm chứng nguyên tắc bảo vệ Fail-Closed trước sự kiện bất thường

```
./c/access_simulator.exe dataset/access_safe_states.csv dataset/access_safe_transitions.csv dataset/ac
cess_safe_init.txt insert_card operator_reset

Log: S0 --(insert_card)--> S1 (Authenticating)
[BI TU CHOI] su kien 'operator_reset' khong hop le tai trang thai 'S1' (fail-closed: giu nguyen trang
thai)
Trang thai cuoi cung: S1 (Khong bi crash, khong nhay trang thai loan, bao ve he thong)
```

5.4. Mô hình Vá Lỗi Toàn Diện (FIXED Model)

Bản vá `access_system_model_FIXED.json` khắc phục triệt để:

1. Loại bỏ trạng thái lỗi S5. Chuyển tiếp timeout tại S1 được đưa về S0 an toàn (`timeout_reset`).
2. Sửa nối sự kiện `tamper_sensor` từ S0 trỏ chuẩn xác đến S4 (TamperDetected_Alarm).
3. Bổ sung thuộc tính P4 (AG !(unlocked AND alarm)) và P5 (EF tamper) nhằm đảm bảo S4 luôn đạt tới được, triệt tiêu hoàn toàn Vacuous Truth.

Chạy kiểm chứng trên mô hình FIXED thu được kết quả tuyệt đối **5/5 thuộc tính PASS**, không còn cảnh báo Vacuous Pass.

6. BỘ KIỂM THỬ TỰ ĐỘNG (AUTOMATED TEST SUITE)

Bộ kiểm thử tự động `tests/test_kripke.py` gồm 8 test case độc lập kiểm tra toàn diện cả Python và C:

STT	Tên Test Case	Mục tiêu kiểm thử	Kết quả
1	test_safe_model_all_properties_hold	Xác nhận mô hình SAFE thỏa mãn P1, P2, P3	PASS
2	test_buggy_model_violates_p1	Xác nhận mô hình BUGGY bắt đúng lỗi P1 và trace S5	PASS
3	test_buggy_model_reachability_still_holds	Xác nhận tính khả thi P3 vẫn tồn tại trên bản BUGGY	PASS
4	test_c_simulator_detects_runtime_violation	Xác nhận simulator C cảnh báo vi phạm P1 lúc chạy	PASS
5	test_c_simulator_safe_sequence_no_warning	Xác nhận simulator C không cảnh báo sai trên luồng chuẩn	PASS
6	test_fixed_model_all_properties_hold	Xác nhận mô hình FIXED thỏa mãn trọn vẹn 5 thuộc tính	PASS
7	test_vacuous_pass_detected_on_buggy	Xác nhận cơ chế cảnh báo Vacuous Pass trên bản BUGGY	PASS
8	test_c_simulator_fixed_sequence	Xác nhận simulator C chạy đúng các kịch bản của bản FIXED	PASS

Lệnh thực thi: `python tests/test_kripke.py` --> Kết quả: **8/8 test da PASS (100%)**.

7. GIẢI ĐÁP TOÀN DIỆN CÂU HỎI THẢO LUẬN (TRANG 5 - LAB1.PDF)

Câu hỏi 1: Tại sao P2 "PASS" trên bản BUGGY lại không đáng tin cậy?

Trả lời: Thuộc tính P2 được biểu diễn dưới dạng logic kéo theo: `AG (tamper -> alarm)`. Trên mô hình BUGGY, do sự kiện `tamper_sensor` bị cấu hình trỏ nhầm về S0, trạng thái S4 (nơi duy nhất có nhãn `tamper`) trở thành trạng thái không thể đạt tới (Unreachable State). Trong toàn bộ không gian trạng thái thực tế mà thuật toán duyệt qua, tiền đề `tamper` luôn mang giá trị **False**. Theo bảng chân trị logic mệnh đề, biểu thức `False -> alarm` luôn luôn đúng (True) một cách rỗng tuếch (**Vacuous Truth**). Kết quả "PASS" này là một cảnh báo giả nguy hiểm, che giấu sự thật rằng hệ thống đã hoàn toàn mất khả năng phản ứng với hành vi cạy phá cửa.

Câu hỏi 2: Nếu thêm một trạng thái mới có cả hai nhãn unlocked và alarm đồng thời, điều gì xảy ra với các thuộc tính hiện có? Cần thêm thuộc tính gì để phát hiện?

Trả lời:

- **Đối với P1 (AG !(unlocked AND NOT authenticated)):** Nếu trạng thái mới này có kèm nhãn `authenticated` thì P1 vẫn PASS; nếu không có nhãn này thì P1 sẽ FAIL.
- **Đối với P2 (AG (tamper -> alarm)):** P2 vẫn PASS vì nhãn `alarm` vẫn đúng.
- **Đối với P3 (EF unlocked):** P3 vẫn PASS vì trạng thái unlocked vẫn có thể đạt tới.
- **Nguy cơ bảo mật và giải pháp:** Việc cửa vừa mở khóa cho người vào vừa hú còi báo động trộm cướp là một mâu thuẫn trạng thái nguy hiểm trong thực tế. Các thuộc tính cũ không phát hiện được sự bất thường này. Do đó, cần bổ sung thuộc tính loại trừ lẫn nhau (Mutual Exclusion Safety):
  `P4: AG !(unlocked AND alarm)`
  Khi có P4, hệ thống sẽ lập tức gắn cờ vi phạm nếu xuất hiện trạng thái xung đột này.

Câu hỏi 3: So sánh chi phí tính toán của thuật toán BFS O(|S| + |R|) với phương pháp kiểm chứng bằng liệt kê vét cạn khi số trạng thái |S| rất lớn?

Trả lời:

- **Thuật toán BFS duyệt trạng thái tường minh (Explicit State Space Exploration):** Có độ phức tạp thời gian tuyến tính **O(|S| + |R|)** và bộ nhớ **O(|S|)** nhờ sử dụng tập đánh dấu `visited`. Thuật toán duyệt qua mỗi trạng thái và cạnh đúng một lần, đồng thời đảm bảo tìm ra vết phản ví dụ ngắn nhất. Tuy nhiên, khi số lượng biến và tiến trình tăng lên, số trạng thái bùng nổ theo cấp số nhân (|S| = 2^n), dẫn tới hiện tượng **Bùng nổ không gian trạng thái (State Space Explosion)** làm tràn RAM.
- **Liệt kê vét cạn mọi đường đi (Exhaustive Path Enumeration):** Có độ phức tạp hàm mũ **O(b^d)** (với b là hệ số rẽ nhánh, d là độ sâu), và sẽ lặp vô hạn nếu đồ thị có chu trình lặp (cycles). Phương pháp này hoàn toàn không khả thi trên thực tế.
- **Giải pháp khi |S| cực lớn:** Trong thực tế công nghiệp khi |S| > 10^8, người ta sử dụng các kỹ thuật biểu diễn hình thức tiên tiến: **Symbolic Model Checking** (dùng Đồ thị quyết định nhị phân BDD) hoặc **Bounded Model Checking (BMC)** dựa trên các bộ giải SAT/SMT (như Z3 Solver được áp dụng ở Lab 5 và Lab 6) để kiểm chứng mà không cần nạp toàn bộ đồ thị trạng thái vào bộ nhớ.

8. TỔNG KẾT BÀI THỰC HÀNH

Bài thực hành Lab 1 đã hoàn thành trọn vẹn và chuẩn chỉ tất cả các yêu cầu đề ra:

1. Xây dựng đầy đủ cấu trúc Kripke lý thuyết và cài đặt thành công bộ kiểm chứng BFS trong Python với tính năng tự động bắt lỗi Chân lý rỗng (Vacuous Truth).
2. Cài đặt chương trình mô phỏng C an toàn bộ nhớ, hiện thực hóa nguyên tắc Fail-Closed và kiểm tra cảnh báo an toàn lúc chạy khớp 100% với mô hình hình thức.
3. Bóc trần lỗ hổng logic race-condition của mô hình BUGGY và thiết kế bản vá FIXED đạt chuẩn an toàn 5/5 thuộc tính.
4. Xây dựng bộ kiểm thử tự động 8/8 test case đạt tỷ lệ thành công 100%, kèm theo 8 tệp nhật ký thực nghiệm (logs) đầy đủ làm bằng chứng minh chứng.